What’s Next?

Building on this knowledge, you can build any native screen or interaction you want! You could swap out the map and render HealthKit data. Or convert the list of hikes to native elements and cache it for offline access. The list goes on.

But with every new native feature, it’s important to understand the trade-offs. SwiftUI screens and the coordinating code are much more expensive to build and maintain. And as we reviewed earlier, they should only be reserved for the most critical pieces of your app.

Up next, we’ll tackle the Android side of this—rendering a native screen with Jetpack Compose. But if you’re all in on iOS right now, feel free to skip the next chapter and jump right to Chapter 7, *Build iOS Bridge Components with Swift*. There, we’ll upgrade individual *components* to native, without committing to an entire SwiftUI screen.

Footnotes

[28] https://developer.apple.com/xcode/swiftui/

[29] https://developer.apple.com/documentation/swiftui/uihostingcontroller

[30] https://developer.apple.com/documentation/swiftui/migrating-from-the-observable-object-protocol-to-the-observable-macro

Copyright © 2025, The Pragmatic Bookshelf.